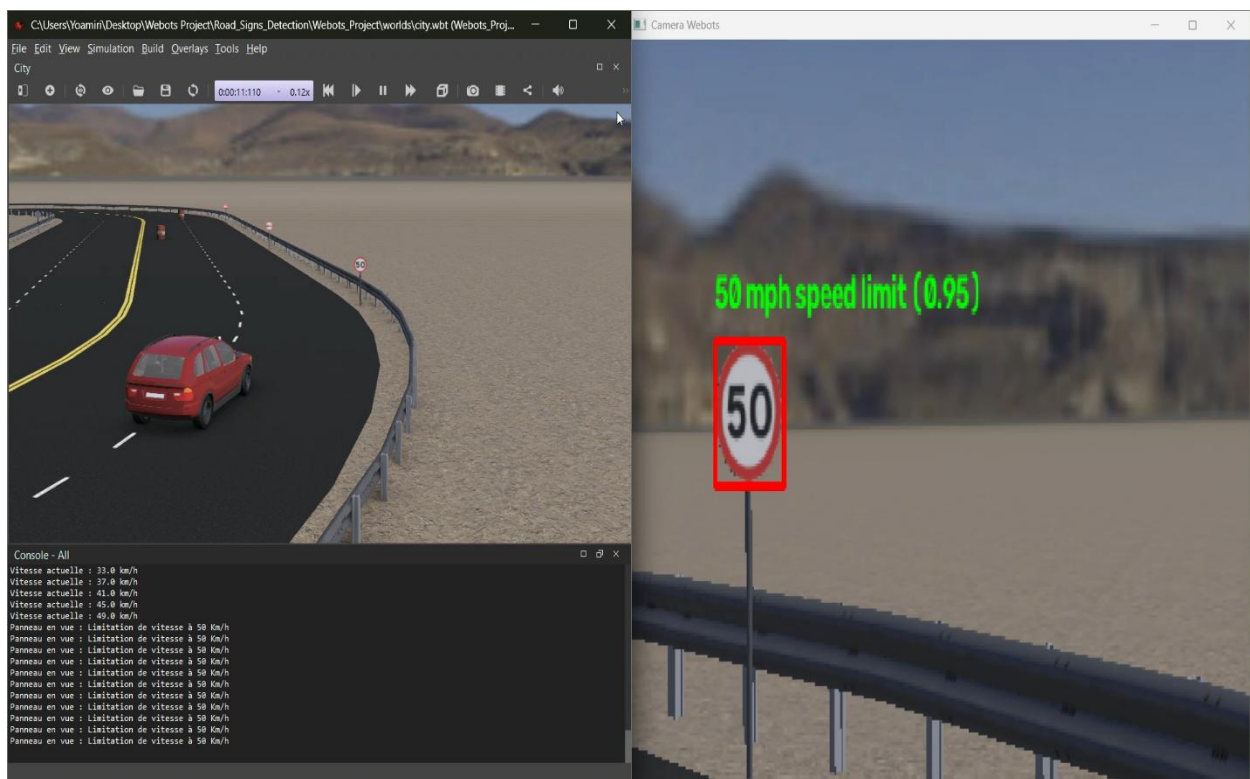




1. Objectifs du Projet

Ce projet technique vise la conception, l'entraînement et l'intégration d'un système avancé d'assistance à la conduite (ADAS) pour un véhicule autonome évoluant dans l'environnement de simulation **Webots**. L'objectif primordial est de doter le véhicule d'une capacité de perception visuelle en temps réel afin d'identifier, de classer et de réagir de manière autonome aux panneaux de signalisation routière rencontrés sur son trajet.

Pour relever ce défi de perception et de contrôle avec de fortes contraintes de temps réel, une architecture hybride originale a été adoptée. Elle couine un algorithme de prétraitement et de filtrage géométrique ultra-rapide basé sur **OpenCV** (extraction de régions d'intérêt par détection de contours polygonaux) avec un réseau de neurones convolutif de pointe **YOLOv8n** entraîné sur un jeu de données spécifique de 29 classes de panneaux routiers. Les résultats démontrent une excellente précision de classification (mAP50 supérieur à 97 %) et un pilotage actif réactif assurant le freinage d'urgence face aux obstacles et le respect automatique des limitations de vitesse.

2. Architecture Globale du Système

L'architecture du système est divisée en quatre sous-systèmes interdépendants fonctionnant en boucle fermée au sein du script contrôleur principal de Webots :

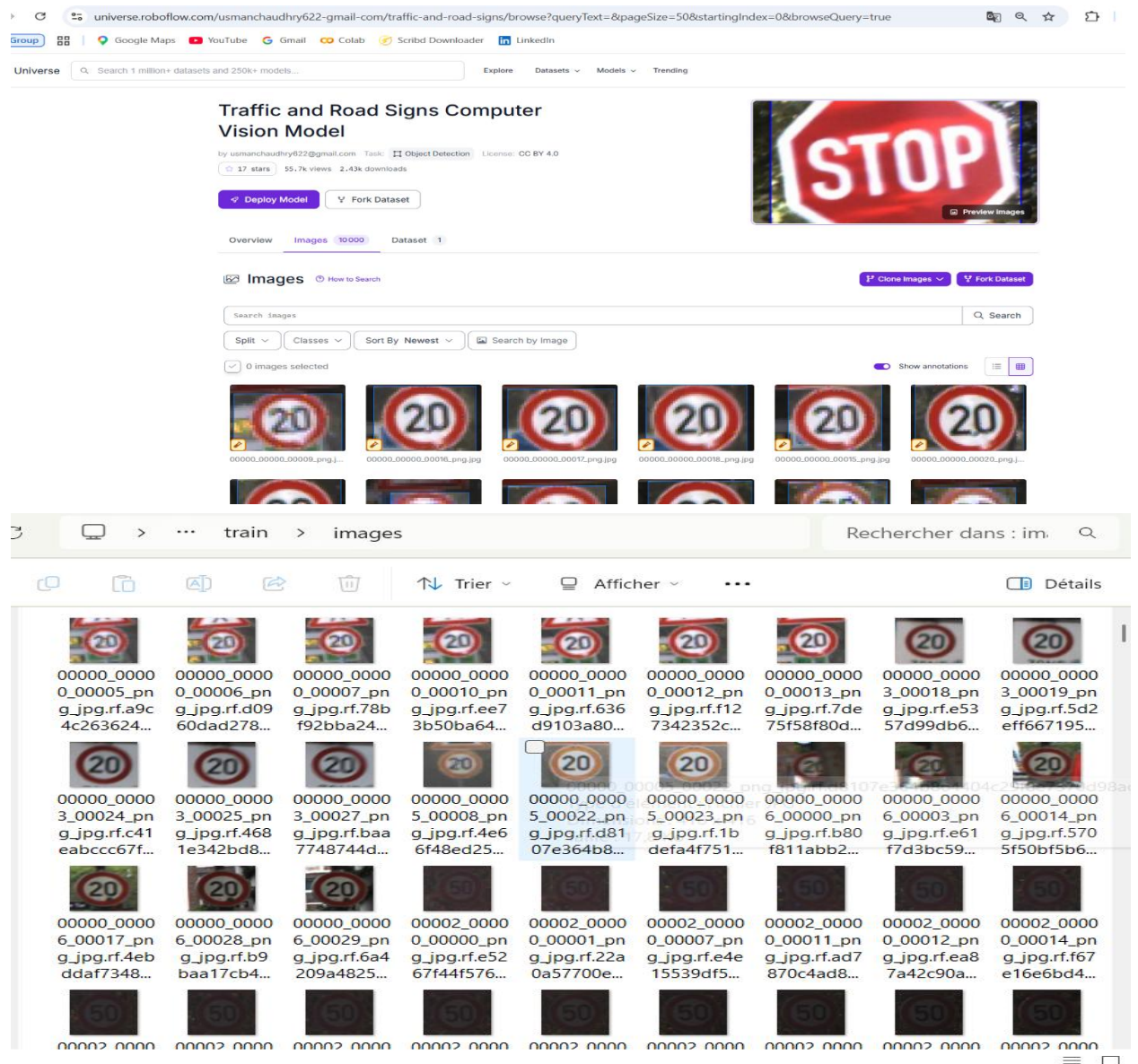
- **Module de Capture & Simulation (Auto_Controller.py)** : Gestion du pas de temps de simulation («timestep»), lecture du capteur caméra embarqué («driver.getDevice("camera")») et acquisition du flux vidéo brut au format RGBA converti en matrice NumPy BGR.
- **Module d'Attention & Prétraitement Visuel (OpenCV ROI : Sign_Detector.py)** : Extraction du tiers central de l'image, application d'un flou gaussien, détection des arêtes de Canny et sélection géométrique des contours candidats (triangles et polygones à 6 côtés ou plus) afin de ne soumettre au réseau de neurones que les zones d'intérêt pertinentes.
- **Module d'Inférence Deep Learning (YOLOv8n)** : Classification et localisation précises des panneaux à partir des vignettes recadrées («crop»), avec un seuil de confiance strict fixé à 0.75 pour éliminer les faux positifs.
- **Module de Décision & Contrôle Actif (ADAS Actuators)** : Traduction sémantique des étiquettes via le module «Fr_Classifier», et modulation dynamique des commandes du véhicule : régulation de la vitesse de croisière («setCruisingSpeed»), application de l'intensité de freinage («setBrakeIntensity») et gestion de l'angle de braquage («setSteeringAngle»).

3. Jeu de Données (Dataset) et Entraînement du Modèle YOLOv8

Le modèle de détection d'objets a été entraîné à l'aide de la bibliothèque **Ultralytics YOLOv8** dans un environnement cloud Google Colab équipé d'un processeur graphique NVIDIA Tesla T4.

Le jeu de données utilisé, intitulé *Traffic and Road Signs* (hébergé sur Roboflow), contient un total de **10000 images annotées** réparties comme suit :

- **Ensemble d'entraînement (Train)** : 7 092 images (servant à l'apprentissage des poids du réseau).
- **Ensemble de validation (Val)** : 1 884 images (servant à l'évaluation des performances et au réglage des hyperparamètres).
- **Ensemble de test** : 1 024 images



Le jeu de données couvre exactement **29 classes** distinctes de panneaux de signalisation, qui ont été regroupées sémantiquement dans le tableau ci-dessous :

Catégorie Fonctionnelle	Classes YOLO Identifiées (Nom d'origine)	Action ADAS Associée
Arrêt & Interdiction	Stop_Sign, No Entry, No_Over_Taking, Overtaking by trucks is prohibited, Truck traffic is prohibited	Freinage d'urgence automatique (AEB) à 0 km/h
Limitations de Vitesse	Speed Limit 20 KMPH, Speed Limit 30 KMPH, 50 mph speed limit, End of all speed and passing limits	Régulation active (ISA) par paliers de -5 à -10 km/h
Dangers & Avertissements	Attention Please-, Beware of children, CYCLE ROUTE AHEAD WARNING, Dangerous Left Curve Ahead, Pedestrian Crossing,	Alerte visuelle / Préparation à la réduction de vitesse
Obligations & Priorités	Give Way, Go Straight or Turn Right, Keep-Left, Round-About, Straight Ahead Only, Traffic_signal, Turn left ahead, Turn right ahead	Assistance à la navigation & avertissement conducteur

Configuration de l'Entraînement et Dynamique de Convergence :

Le réseau sélectionné est **YOLOv8n Nano**, caractérisé par son excellente compacité (130 couches, 3 016 503 paramètres pour 8,2 GFLOPs), un atout décisif pour l'exécution embarquée en temps réel. Le processus d'apprentissage s'est déroulé sur **100 époques** avec des images redimensionnées à 416x416 pixels («imgsz=416»), une taille de lot de 16 («batch=16») et un optimiseur automatique **MuSGD** («lr=0.01», «momentum=0.9»). L'enrichissement des données (Data Augmentation) a été assuré par Albumentations via des transformations de flou («Blur», «MedianBlur»), conversion en niveaux de gris pondérés («ToGray») et égalisation d'histogramme adaptative («CLAHE»).

```

from ultralytics import YOLO

model = YOLO('yolov8n.pt') # .yaml au lieu de .pt crée un modèle vide

# Entraînement
results = model.train(
    data='/content/datasets/data.yaml',
    epochs=100,
    imgsz=416,
    device=0
)

# Informations sur l'entraînement
val: Fast Image access (img: 0.020 s, read: 430.22147 MB/s, size: 13.1 KB)
val: Scanning /content/datasets/valid/labels... 1884 Images, 0 Backgrounds, 0 corrupt: 100% — 1884/1884 1.3KIt/s 1.5s
val: New cache created: /content/datasets/valid/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr' and 'momentum' automatically...
optimizer: PyTorch(lr=0.01, momentum=0.9) with parameter groups 57 weight(decay=0.0), 64 weight(decay=0.0001), 63 bias(decay=0.0)
Plotting labels to /content/runs/detect/train/labels.jpg...
Image size 416 train, 416 val
Using 2 dataloader workers
Logging results to /content/runs/detect/train
Starting training for 100 epochs...

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/100    0.9650    0.9324    1.379    1.27    9    416: 100% — 444/444 3.01t/s 1:50
Class    Images  Instances  Box(P  R  mAP50  mAP50-95): 100% — 50/50 6.11t/s 9.7s
all      1884    1886    0.505  0.609  0.493  0.382

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
2/100    1.176    0.8973    1.806    1.177    12   416: 100% — 444/444 4.81t/s 1:33
Class    Images  Instances  Box(P  R  mAP50  mAP50-95): 100% — 50/50 7.11t/s 8.3s
all      1884    1886    0.715  0.728  0.704  0.554

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
3/100    1.170    0.895    1.816    1.175    13   416: 100% — 444/444 5.21t/s 1:25
Class    Images  Instances  Box(P  R  mAP50  mAP50-95): 100% — 50/50 6.31t/s 9.4s
all      1884    1886    0.718  0.796  0.751  0.603

```

```

Epoch    GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
91/100    1.176    0.4734    0.1093    1.019    4    416: 100% — 444/444 5.51t/s 1:20
Class    Images  Instances  Box(P  R  mAP50  mAP50-95): 100% — 50/50 5.81t/s 10.1s
all      1884    1886    0.944  0.948  0.934  0.792

100 epochs completed in 2.638 hours.
Optimizer stripped from /content/runs/detect/train/weights/last.pt, 6.2MB
Optimizer stripped from /content/runs/detect/train/weights/best.pt, 6.2MB

Validating /content/runs/detect/train/weights/best.pt...
Ultralytics 8.0.99 d Python 3.12.13 torch-2.11.0+cu121 CUDA0 (Tesla T4, 14913MB)
Model summary (fused): 73 layers, 3,011,303 parameters, 0 gradients, 8.1 GPU ops
Class    Images  Instances  Box(P  R  mAP50  mAP50-95): 100% — 50/50 4.61t/s 12.8s
all      1884    1886    0.957  0.963  0.959  0.799
-Road narrows on right 54 54 0.992 1 0.995 0.989
50 mph speed limit 42 42 0.957 1 0.976 0.945
Attention Please- 117 117 0.997 1 0.995 0.957
Beware of children 54 54 0.997 1 0.995 0.89
CYCLE ROUTE AHEAD WARNING 42 42 0.992 1 0.995 0.863
Dangerous Left Curve Ahead 42 42 0.991 1 0.995 0.869
Dangerous Right Curve Ahead 72 72 0.982 1 0.995 0.811
End of all speed and passing limits 48 48 0.992 1 0.995 0.812
Give Way 107 107 0.997 1 0.995 0.837
Go Straight or Turn Right 79 79 0.995 1 0.995 0.867
Go straight or turn left 42 42 0.99 1 0.995 0.864
Keep-Left 64 64 0.994 1 0.995 0.831
Keep-Right 82 82 0.996 1 0.995 0.817
Left Zig Zag Traffic 64 64 0.999 1 0.995 0.865
No Entry 83 83 0.995 1 0.995 0.803
Overtaking by trucks is prohibited 47 47 0.992 1 0.995 0.827
Pedestrian Crossing 47 47 0.947 1 0.995 0.869
Round-About 67 67 0.995 1 0.995 0.866
Slippery Road Ahead 103 103 0.987 1 0.985 0.901
Speed limit 20 km/h 48 48 0.732 1 0.948 0.752
Speed limit 30 km/h 79 79 1 0 0 0
Stop-Sign 32 32 0.969 0.983 0.994 0.896
Straight Ahead Only 63 63 0.994 1 0.995 0.862
Traffic-signal 4 6 0.786 0.833 0.786 0.404
Truck traffic is prohibited 84 84 0.995 1 0.995 0.934
Turn left ahead 94 94 0.992 0.995 0.995
Turn right ahead 80 80 0.973 1 0.993 0.867
Uneven Road 77 77 0.994 1 0.995 0.886
Speeds: 0.3ms preprocess, 1.2ms inference, 0.0ms loss, 1.8ms postprocess per image
Results saved to /content/runs/detect/train

```

L'analyse des journaux de convergence révèle une progression exceptionnelle des métriques de précision :

- **Phase d'amorçage (Époques 1 à 10) :** Le score mAP50 progresse extrêmement rapidement, passant de 0.493 (époque 1) à 0.704 dès l'époque 2, pour atteindre 0.878 à l'époque 10. Les pertes de localisation («box_loss») et de classification («cls_loss») chutent massivement.
- **Phase de stabilisation (Époques 11 à 50) :** Le réseau franchit le seuil de 0.90 mAP50 à l'époque 12 (0.904) et se stabilise de manière constante au-dessus de 0.95 à partir de l'époque 45, avec un mAP50-95 avoisinant 0.787.
- **Phase de maturation optimale (Époques 51 à 100) :** Les performances atteignent leur zénith avec un pic de précision de **0.977 en mAP50** à l'époque 63 (rappel R = 0.943, précision P = 0.977, mAP50-95 = 0.799). Le modèle final («best.pt») garantit ainsi une détection extrêmement fiable et quasiment exempte de faux négatifs sur le jeu de validation.

4. Pipeline d'Optimisation par Vision par Ordinateur (ROI OpenCV)

Afin d'éviter d'exécuter l'inférence du réseau de neurones sur des zones d'arrière-plan inutiles (ciel, chaussée immédiate, décor lointain), la classe «Detector» (fichier «sign_detector.py») implémente un filtre de présélection géométrique extrêmement astucieux. Cette méthode, implémentée dans la fonction « find_potential_signs(frame) », fonctionne selon la séquence d'opérations suivante :

1. **Focalisation Spatiale (Cropping de Zone)** : Lors de la phase de détection («detect_and_draw»), l'image globale de la caméra est d'abord recadrée pour se concentrer exclusivement sur le tiers central. C'est dans ce corridor spatial que les panneaux routiers apparaissent naturellement avec une résolution optimale pour le pilotage.
2. **Filtrage du Bruit et Détection d'Arêtes** : La sous-image est convertie en niveaux de gris («cv2.cvtColor»), puis lissée par un filtre de Gauss («cv2.GaussianBlur») avec un noyau de 5×5 pour supprimer le bruit haute fréquence. L'opérateur de Canny («cv2.Canny») est ensuite appliqué avec des seuils d'hystérésis stricts de 80 et 200 afin d'extraire les contours nets.
3. **Approximation Polygonale et Sélection Géométrique** : L'algorithme extrait les contours fermés «cv2.findContours» et écarte tout contour dont l'aire est inférieure à 400 pixels carrés «area < 400». Une approximation polygonale «cv2.approxPolyDP» est réalisée avec une tolérance égale à 4 % du périmètre «0.04 * perimetre». Le critère de sélection géométrique conserve uniquement les polygones ayant exactement **3 côtés** (panneaux triangulaires de danger ou cédez-le-passage) ou **au moins 6 côtés** (panneaux octogonaux de Stop, ou panneaux circulaires de limitation et d'interdiction dont la courbure est approximée par un polygone à nombreux côtés).
4. **Inférence YOLO Targetée** : Pour chaque zone d'intérêt géométriquement validée, un rectangle englobant «boundingRect» est calculé en ajoutant une marge de sécurité de 5 pixels «marge = 5». Seule cette vignette recadrée («crop») est transmise au modèle YOLO redimensionnée à une résolution fine de 256x256 «imgsz=256». Si la probabilité prédite dépasse **0.75**, le panneau est validé, encadré en rouge sur l'affichage et ajouté à la liste des détections.

5. Intégration du Contrôle Autonome et Logique ADAS

Le script de contrôle du véhicule «controller.py» lie la perception à l'action. Pour préserver les ressources de calcul et maintenir une simulation fluide sous Webots, la détection visuelle n'est déclenchée que **toutes les 5 trames de simulation** (pour l'évitement de blocage de simulation). Entre deux cycles d'analyse visuelle, le conducteur conserve la possibilité d'un contrôle manuel de la vitesse et de la direction via les touches directionnelles du clavier (flèches HAUT/BAS pour l'accélération/décélération, DROITE/GAUCHE pour l'angle de braquage).

Dès qu'un panneau est reconnu par la caméra, il est d'abord traduit en sémantique française par le

module de classification textuelle «Fr_Classifier(panneau)», puis le contrôleur active l'une des deux logiques de sécurité d'assistance active :

5.1 Freinage d'Urgence Automatique (AEB)

Lorsque l'algorithme identifie un panneau d'arrêt formel («Stop_Sign») ou d'accès interdit («No Entry»), et si la vitesse instantanée du véhicule est positive («speed > 0.0»), un protocole d'arrêt d'urgence s'exécute instantanément. La consigne de vitesse de croisière est forcée à 0 km/h et l'actionneur de frein est saturé à son intensité maximale :

```
if panneau in ['Stop_Sign', 'No Entry']:
    if speed > 0.0:
        print(f"\nDANGER ({Sign}) : Freinage d'urgence automatique !")
        speed = 0.0
        driver.setCruisingSpeed(0.0)
        driver.setBrakeIntensity(1.0)
        print("Vous pouvez accélérer pour repartir.\n")
```

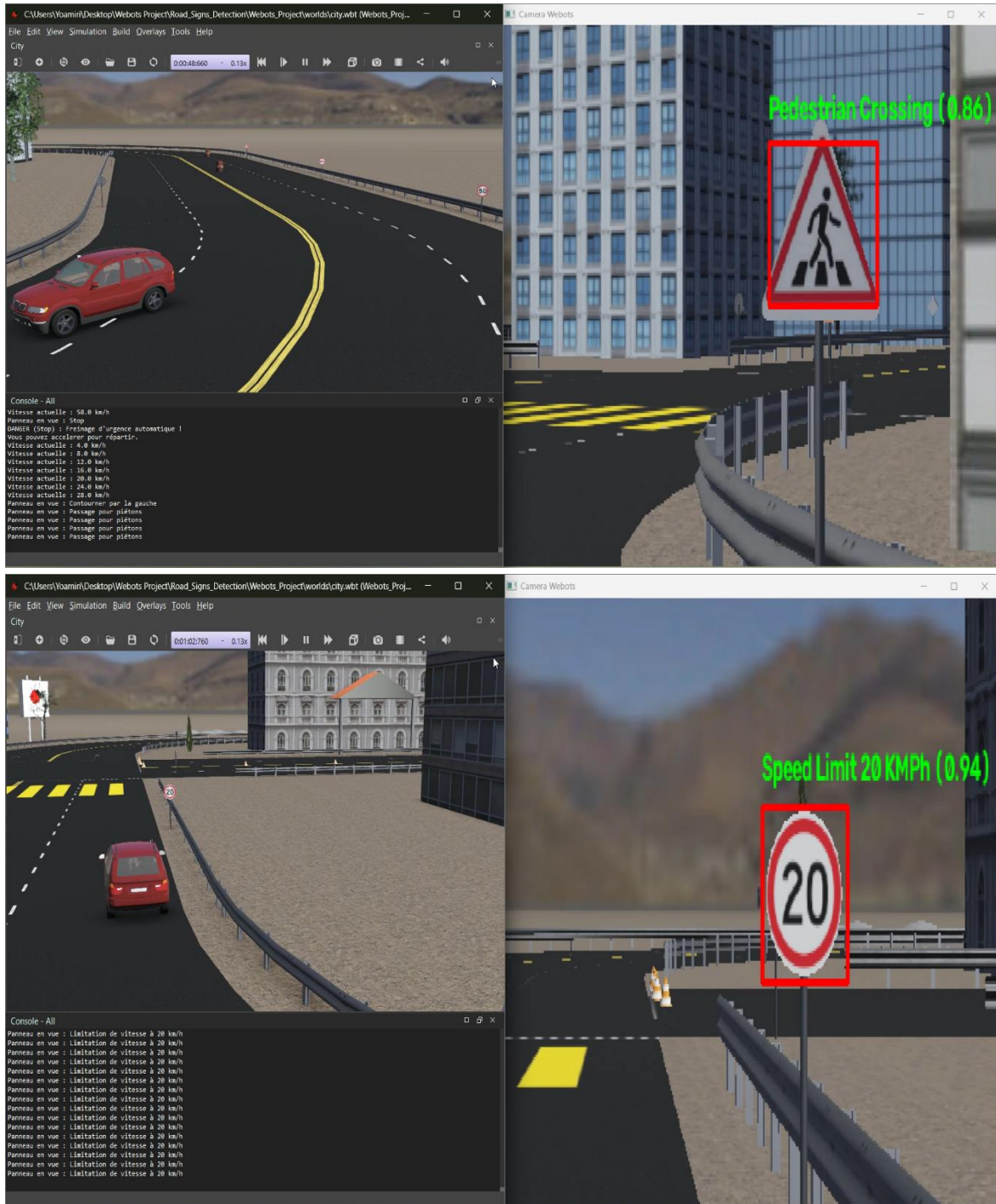
5.2 Adaptation Intelligente de la Vitesse (ISA - Intelligent Speed Assistance)

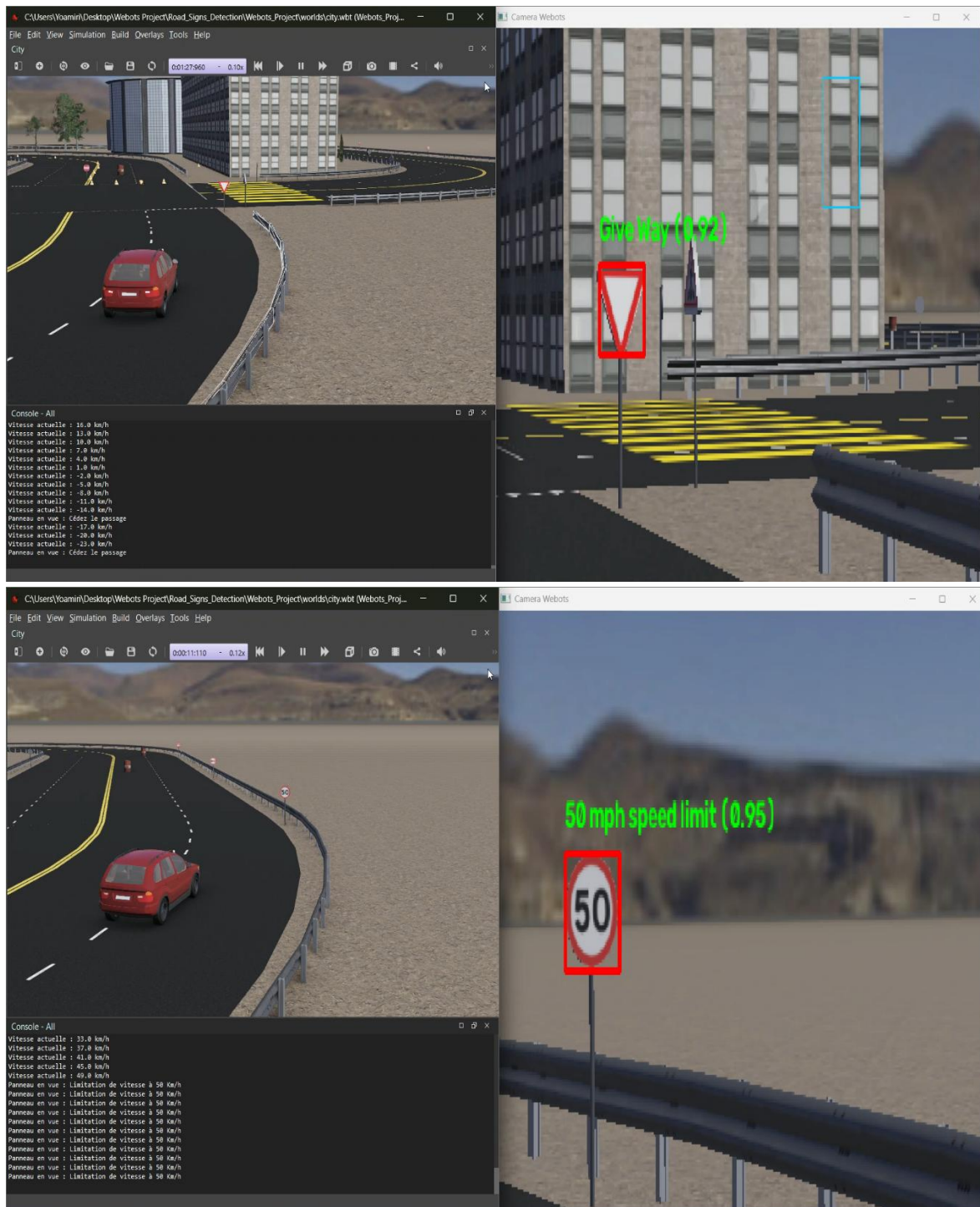
Lorsque des panneaux de limitation de vitesse sont détectés «50 mph speed limit», «Speed Limit 20 KMPH», «Speed Limit 30 KMPH», le système vérifie si la vitesse de croisière actuelle excède le seuil réglementaire. Si une infraction est constatée, une boucle de décélération est amorcée par décréments stricts afin d'aligner le véhicule sur la vitesse autorisée sans action requise de la part de l'utilisateur :

```
if panneau == "Speed Limit 30 KMPH":
    while speed > 30:
        speed -= 10
        driver.setCruisingSpeed(speed)
        print("Freinage! Vitesse élevée > 30")
        print(f"Vitesse actuelle : {speed:.1f} km/h")
```

6. Bilan Technique, Résultats, Limites et Perspectives d'Évolution

Résultats :





L'aboutissement de ce projet met en lumière plusieurs succès majeurs d'ingénierie robotique, tout en permettant d'identifier des axes d'optimisation clairs pour les versions futures de l'architecture :

Domaine d'Analyse	Points Forts Constatés (Réalisations)	Limites & Perspectives d'Amélioration
Performance de Perception	Le modèle YOLOv8n atteint un score mAP50 remarquable de 97.7 % sur 29 classes complexes. L'extraction par ROI OpenCV réduit drastiquement le temps d'inférence en se focalisant sur des vignettes utiles.	Le critère géométrique strict (3 côtés ou >6 côtés) écarte par construction les panneaux carrés ou rectangulaires (ex : indications de direction ou de priorité ponctuelle). Un élargissement de ces règles géométriques est recommandé.
Contrôle et Dynamique Webots	L'intégration d'une fréquence de détection temporisée (1 trame sur 5) garantit une excellente fluidité de simulation sans ralentissement du thread principal du moteur physique.	Les boucles «while » dans le script de régulation s'exécutent de manière synchrone dans un seul tour de boucle Python, provoquant un changement de consigne instantané au lieu d'un freinage lissé dans le temps. L'introduction d'un régulateur PID ou d'une machine d'états finis asynchrone améliorera le réalisme physique.
Robustesse Environnementale	L'utilisation d'augmentations d'images (flou et contraste CLAHE) rend le modèle résilient aux variations d'éclairage standard en simulation.	Il serait pertinent de tester la robustesse du système sous des conditions dégradées simulées dans Webots (brouillard dense, pluie, conduite de nuit avec phares) et d'évaluer une transition vers un modèle plus capacitif comme YOLOv8s ou YOLOv11.

7. Conclusion Générale

Ce projet valide avec succès la faisabilité et la pertinence d'un système ADAS embarqué couplant vision classique par ordinateur et intelligence artificielle de pointe au sein d'une simulation robotique réaliste. Le pipeline développé apporte une solution élégante au compromis précision/vitesse dans les systèmes de transport intelligents : en exploitant la rapidité des filtres OpenCV pour focaliser l'attention du réseau YOLOv8n, le véhicule autonome est capable de percevoir son environnement, de comprendre la signalisation routière (29 classes) et d'adopter un comportement de conduite sécuritaire en temps réel. Cette base technique solide constitue un tremplin idéal pour des développements futurs vers la conduite autonome de niveau 3 et 4.